CampusDesk: exact presentation and demonstration script

Prepared for Aryan (AJ5831A). Estimated delivery: 15–20 minutes. This script is for the updated ten-slide CampusDesk deck. It credits the shared ResolveDesk foundation and AI assistance.

Current state: the AWS cluster has been destroyed. The default demonstration below uses the running local application and clearly labelled recorded cloud evidence. It does not require AWS login or create billable resources. The existing video records the earlier real cloud session; the updated slides include final cleanup results that became available afterward.

Before presenting

1. Open a terminal in the CampusDesk project directory. On the prepared machine, use the command below. On another machine, use the directory where you cloned the repository.

```
cd /Users/dibyochakraborty/campusdesk-devsecops
docker compose up -d --build --wait
docker compose ps
```

2. Open `demo/slides.pdf` in presentation mode. Open this script on a second screen or print `demo/demo-script.pdf`.

3. Open http://localhost:3002 in a browser. The heading should say **Campus IT queue**. Open http://localhost:8001/docs in another tab.

4. Open the repository: https://github.com/AJ5831A/campusdesk-devsecops.

5. Open the completed CI run: https://github.com/AJ5831A/campusdesk-devsecops/actions/runs/38906213468.

6. Open `demo/evidence/` in the file browser. Keep `demo/walkthrough.mp4` available as the recording of the earlier live cloud session.

7. Use only the synthetic values below. If a previous rehearsal left the same ticket, identify that rehearsal ticket by its subject before deleting it or use a new subject suffix.

Opening words — say exactly:

> Hello, I am Aryan, and this is CampusDesk for campus IT support. I will first explain the design, then demonstrate the application locally and show recorded evidence from its completed cloud deployment. The temporary AWS cluster has already been destroyed. This implementation shares the credited ResolveDesk foundation and was prepared with AI coding assistance. I will distinguish what is running now from what was recorded earlier.

Slide 1 — CampusDesk: support across campus

Show: slide 1. Point to the location and category requirements.

Say:

> CampusDesk tracks technology problems across classrooms, labs and campus accounts. Its distinct workflow requires a campus location and classifies tickets as hardware, software, network, account or other. That information helps the support team understand where the incident happened and who should handle it. The project shares an openly credited foundation with ResolveDesk; it is not presented as independently written from scratch.

Slide 2 — From a lab incident to a resolution

Show: slide 2. Follow the incident from creation to resolution.

Say:

> The example I will demonstrate is a projector losing its HDMI signal in Science block, Lab three hundred and one. A ticket records the requester, location, category, priority and assigned support team. It moves from Open to In progress to Resolved. Search includes category and location, so a technician can quickly find incidents in a particular lab. This classroom demo uses synthetic data, has no authentication and does not send email.

Slide 3 — Campus fields have their own contract

Show: slide 3. Point to the form, API validation and migration responsibilities.

Say:

> The implementation separates the interface, validation, workflow operations and storage. The React form collects the campus fields, and the table displays them. Pydantic validates the category and requires a nonblank location. Route handlers call service functions, and SQLAlchemy writes the data to PostgreSQL. Alembic migration zero zero zero three adds the campus fields without deleting existing tickets. Tests cover these additions as well as the shared workflow.

Slide 4 — One shared cluster, separate campus data

Show: slide 4. Trace Browser to React and Nginx, then FastAPI and PostgreSQL. Trace the delivery path below it.

Say:

> The browser talks to the React and Nginx frontend, which forwards API requests to FastAPI. The API uses PostgreSQL. Locally, Compose runs the same three services. The application is on port three thousand and two, and the API is on eight thousand and one. During the recorded AWS deployment, CampusDesk shared one EKS cluster with ResolveDesk, but used the campusdesk namespace and separate database credentials and storage. GitHub Actions tested and published images, and ArgoCD applied the release recorded in Git.

Slide 5 — A tested image reaches the registry

Show: slide 5. Keep the final completed run tab ready; do not claim a new run is occurring.

Say:

> Thirty-one backend tests and four frontend tests passed. The pipeline also built the production frontend. It built commit-tagged container images, blocked high and critical Trivy findings, loaded the scanned images into a temporary Kind cluster and smoke-tested the Helm deployment before publishing to GHCR. The final published baseline run is three eight nine zero six two one three four six eight, for commit one b d four eight nine six seven. I will open that completed run rather than describe an old result as a new push.

Slide 6 — Quota recovery changed the network plan

Show: slide 6. Emphasize the difference between a working replacement deployment and an unfinished apply operation.

Say:

> The first design used private workers and a NAT gateway, but the account had exhausted its regional Elastic IP quota. The approved temporary recovery used public-subnet workers, blocked unsolicited public inbound traffic and restricted the Kubernetes API to the operator address. The replacement workers successfully served both applications. However, the failed original node group took too long to disappear. The recovery Terraform apply was gracefully interrupted while waiting for that deletion. I am not calling that apply a clean success; the working runtime and the later successful destroy are documented separately.

Slide 7 — The campus workload was verified

Show: slide 7. Point to replicas, migration and storage. State clearly that these are recorded results.

Say:

> The recorded Kubernetes evidence shows two backend replicas, two frontend replicas and a completed migration Job. PostgreSQL had its own bound five-gigabyte volume in the campusdesk namespace. Probes, resource limits and the horizontal pod autoscaler were configured, and CPU metrics were available. This proves what ran during that session. The cluster no longer exists today, so the default demonstration uses the local application and the saved cloud evidence rather than trying to run kubectl against a deleted cluster.

Slide 8 — GitOps was more than a YAML file

Show: slide 8. Point to the recorded ArgoCD status and image tag.

Say:

> ArgoCD read the CampusDesk Helm chart and release values from Git. Its recorded state was Synced and Healthy, with a successful operation. The deployed images used the verified nine a seven a one eight four tag. Later documentation commits did not automatically change that selected application release. The configuration also ignores the backend replica count when comparing desired state, so ArgoCD does not undo scaling decisions made by the horizontal pod autoscaler.

Slide 9 — Check actual metrics, not assumptions

Show: slide 9. Prepare the Prometheus and Grafana evidence images.

Say:

> Prometheus recorded both CampusDesk API endpoints as up. The Grafana dashboard contains actual request-rate and ninety-fifth-percentile latency samples selected from the campusdesk namespace. The server-error panel had no matching series, which is different from measuring a zero error rate. A monitoring pod restart invalidated the earlier Grafana login, so the walkthrough uses a fresh authenticated dashboard recording. Installing a chart or dashboard file alone would not prove any of these runtime results.

Slide 10 — Delivery finished with a cleanup audit

Show: slide 10. Read the final cleanup numbers and the KMS exception.

Say:

> The completed delivery includes thirty-one backend tests, four frontend tests, browser and mobile checks, PostgreSQL integration checks, a slide deck and an eleven-minute-thirty-eight-second

> recorded walkthrough. The shared Terraform destroy removed fifty-nine resources, and all eleven scoped cleanup checks passed. The owned KMS key is scheduled for deletion on the fifteenth of November, twenty twenty-six, because AWS requires a waiting period. That is recorded separately from the removed compute and network resources. The local demonstration remains available. A production service would still need authentication, backups and a recovery plan.

Demonstration 1 — Create a real local campus ticket

Show: http://localhost:3002. Say:

> This is the application running locally now. I will create a synthetic campus incident, change its category and location, resolve it and delete only this demonstration ticket.

Do these exact actions:

1. Click **New ticket**.
2. Set **Subject** to `Projector HDMI signal drops — Aryan demo`.
3. Set **Requester email** to `student@example.com`.
4. Set **Category** to **Hardware**.
5. Set **Campus location** to `Science block, Lab 301`.
6. Set **Description** to `Synthetic demonstration. The projector disconnects during presentations.`
7. Set **Priority** to **High**. Keep **Status** as **Open**.
8. Set **Assigned to** to `Campus IT`.
9. Click **Create ticket**. Wait until the ticket appears in the queue. If an error appears, explain it and retry only after correcting the cause; do not say it saved.

Say after the ticket appears:

> The ticket is now persisted. The queue shows the requester and the campus location and category alongside its priority, status and assignee.

Demonstration 2 — Edit, search, resolve and delete

1. Click the newly created ticket's subject, or its **Edit** action.
2. Change **Category** to **Software**, **Campus location** to `Science block, Lab 302`, and **Status** to **In progress**.
3. Click **Save changes**. Wait for the updated queue row.
4. In **Search tickets**, type `Lab 302`. Show that the demo row matches. Other matching tickets may also appear.
5. Replace the search with `software`. Point out that category is searchable too. Clear the search.
6. Click **In progress** in the left navigation and show the demo ticket. Click **All tickets** before the next step.
7. Edit this same ticket again. Select **Resolved**, then click **Save changes**.
8. Click **Resolved** in the left navigation. Locate this exact demo subject.

9. Click that row's **Delete** action. In the browser confirmation, confirm deletion only for this demo ticket. Verify the row disappears.

Say:

> The changes were saved through the API. Location and category search help campus technicians find relevant incidents. The status queue reflects the workflow, and deletion requires confirmation. I have removed only the synthetic ticket created in this demonstration.

Optional mobile view: resize the browser to approximately 390 pixels wide, or enable a phone viewport in developer tools. Show that each ticket still exposes its location, category, priority, status and actions. Return to desktop width afterward.

Demonstration 3 — Show the API and code

Open: http://localhost:8001/docs. Expand `GET /api/tickets`, click **Try it out**, set `search` to `NETWORK`, click **Execute**, and inspect the response. An empty list is valid when no matching ticket exists.

Say:

> The API exposes the same ticket workflow used by the interface. Inputs are validated before storage, and the list endpoint supports search and pagination. I will show the source files responsible for these behaviors.

Open these files in this order:

1. `frontend/src/components/TicketForm.jsx` — point to the category selector and required location input.

2. `frontend/src/components/TicketTable.jsx` — point to the visible campus/category column.

3. `backend/app/schemas.py` — show `Category`, `TicketCreate.location`, and null-update validation.

4. `backend/app/routes.py` then `backend/app/services.py` — explain that HTTP routing and database operations are separated.

5. `backend/alembic/versions/0003_campus_ticket_fields.py` — show the additive migration.

6. `backend/tests/test_api.py` — show `test_campus_category_location_and_search` and `test_location_is_required`.

Say:

> These are the concrete campus-specific changes: data fields, validation, storage migration, visible form and table controls, search and tests. The shared implementation and AI assistance are credited in the README.

Run these read-only health checks and the synthetic smoke test from the project root:

```
curl --fail http://localhost:8001/health
curl --fail http://localhost:8001/ready
python3 scripts/smoke-api.py --base-url http://localhost:8001
```

Expected: health is `UP`, readiness is `READY`, and the smoke script reports `PASS`. It creates and removes its own uniquely named synthetic ticket.

Optional test rerun: if a backend virtual environment has not been set up in this checkout, create one first. Do not assume a `.venv` was committed.

```
python3 -m venv backend/.venv
backend/.venv/bin/python -m pip install \
  -r backend/requirements-dev.txt
backend/.venv/bin/python -m pytest -q backend/tests
npm ci --prefix frontend
npm test --prefix frontend
npm run build --prefix frontend
```

Expected: 31 backend tests and four frontend tests pass. Package installation needs internet access. For a shorter presentation, show `demo/evidence/local-pytest.png` and `local-frontend-tests.txt`, explicitly calling them recorded output.

Demonstration 4 — Explain the completed CI run

Open: https://github.com/AJ5831A/campusdesk-devsecops/actions/runs/38906213468.

Do: show the successful **test** job. Open **build-scan-publish** and point to image build, Trivy scan, Kind/Helm deployment, smoke test and publication steps. Open `.github/workflows/ci-cd.yml` to connect these steps to their configuration.

Say:

> This is the completed final published-baseline run for commit one b d four eight nine six seven. It is historical evidence, not a new run happening during this presentation. The earlier recorded walkthrough includes a real push and records its pipeline through success. Source images are identified by commit tags, while Helm release values explicitly select the tested deployment version.

Recorded push-and-run evidence:
https://github.com/AJ5831A/campusdesk-devsecops/actions/runs/38902732947. The recording is `demo/walkthrough.mp4`.

Optional: demonstrate a genuine new push

Use this only if you have a real, reviewable improvement to publish. Examples include correcting a documented mistake you found or adding a missing meaningful test. Explain the actual change. Do not create an empty commit or a fake change merely to trigger a run.

```
git status --short
git diff
```

Stage only the file you intentionally changed. The command below illustrates a real README correction; use it only if that is actually your change.

```
git add README.md
git diff --cached
git commit -m "Clarify the verified local demo steps"
git push origin main
```

Open https://github.com/AJ5831A/campusdesk-devsecops/actions, select the new run whose commit matches `git rev-parse HEAD`, and wait for the whole run to finish. A push alone is not a passing pipeline. If no genuine change is ready, use the completed run and the existing recorded push instead.

Demonstration 5 — Show recorded cloud proof honestly

Say before opening any cloud artifact:

> The AWS resources were temporary and are already removed. The following screenshots and logs are evidence captured during the real deployment. I am not claiming that the cluster or these dashboards are running now.

Use these exact files, in order:

1. `demo/evidence/shared-terraform-plan.log` and `terraform-recovery-note.txt`. Explain the Elastic IP quota failure and the approved public-worker recovery.

2. `demo/evidence/shared-aws-vpc-console.png` and `shared-aws-eks-console.png`. Explain that they are actual AWS Console captures from one shared cluster.

3. `demo/evidence/eks-campus-resources.png`. Point to the running replicas, completed migration, campus namespace, ingress and bound persistent volume.

4. `demo/evidence/argocd-campus.png`. Point to **Healthy**, **Synced** and **Sync OK**.

5. `demo/evidence/prometheus-campus-targets.png`. Point to the two CampusDesk API endpoints and **UP** labels.

6. `demo/evidence/grafana-campus.png`. Point to populated request-rate and latency panels. Explain that **No data** on the error panel is not a measured zero.

7. `demo/evidence/terraform-destroy-summary.png`. Read **59 resources destroyed**.

8. `demo/evidence/aws-cleanup-audit.png`. Read **11 passed checks** and the KMS scheduled deletion date.

Say:

> These are separate forms of proof: application behavior, deployment state, monitoring and cleanup. The recovery apply was interrupted while waiting for failed-node-group removal; that is preserved in the log. The later destroy completed successfully. All eleven scoped cleanup checks passed, including removed compute and storage, deleted EKS and VPC resources and empty Terraform state. The KMS key remains scheduled for deletion under the AWS waiting period. No second CampusDesk cluster was provisioned.

Public evidence links:

- Evidence index:
https://github.com/AJ5831A/campusdesk-devsecops/blob/main/demo/evidence/README.md

- Cleanup audit:
https://github.com/AJ5831A/campusdesk-devsecops/blob/main/demo/evidence/aws-cleanup-audit.txt

- Recovery explanation: https://github.com/AJ5831A/campusdesk-devsecops/blob/main/demo/evidence/terraform-recovery-note.txt

- Recorded walkthrough:
https://github.com/AJ5831A/campusdesk-devsecops/blob/main/demo/walkthrough.mp4

Optional fresh cloud demonstration — not the default

A new live AWS walkthrough would require a new deployment, AWS access, quota checks, fresh credentials, explicit ownership of Terraform state, new captures and teardown afterward. This script does not provision it. The CampusDesk Terraform copy is a reference to the shared infrastructure; do not apply it independently against the old shared cluster name.

If a fresh environment has already been deliberately provisioned and its kubeconfig supplied by the operator, the following are read-only inspection commands. Confirm the context points to that new environment before continuing.

```
kubectl config current-context
kubectl get nodes
kubectl get pods,services,ingress,pvc,hpa -n campusdesk
kubectl get application campusdesk -n argocd
```

Use the new deployment's actual ingress address and authorized UI access to demonstrate ArgoCD, Prometheus and Grafana. Do not reuse old port-forward URLs or saved login files as if they still work. Capture a new plan/apply result and a new destroy/audit result. This is a separate run, not an extension of the destroyed session shown in the current evidence.

Closing words

CampusDesk adds a campus-specific incident workflow to a credited shared foundation. I have shown its local behavior, its tested delivery pipeline and recorded proof of the earlier cloud deployment and cleanup. The local app is available for review, while the temporary AWS environment has been removed. The main production gaps are authentication, backups and a recovery plan. Thank you.